Context Engineering

Budgets, compaction, progressive disclosure, subagents, handoffs, and layers of truth: the window is an artifact with a budget

SURVEY | BUILD | GOVERN | MEASURE

CIDM 6333 · Agentic Systems Design and Development
Survey: Claude Code, Codex, Cursor, and OpenCode context surfaces · Build: a budget meter, a compactor with a keep list, a handoff, and an append-only ledger (lab, self-guided)

Two hours, six moves

The run of show; the lab is yours

1	0:02	**Framing**	The window is an artifact with a budget; two rules; the Sortie 6 claim and what falsifies it
2	0:10	**Spine**	Window contents; budget; instrument; compaction; disclosure and subagents; handoffs; layers of truth; mistakes
·	0:55	**Break**	Five minutes
3	1:00	**Teardowns**	Claude Code, Codex, Cursor, OpenCode, and Anthropic's long-running harness, read for mechanism; a live Design Analysis
4	1:30	**Reference design**	State doctrine and the three layers; handoffs and the rulings store; the stale-clone incident; orientation hooks
5	1:45	**Lens and card**	Weak, sound, exceptional for Sortie 6; Flight Card 6; the lab and the Sprint
6	2:00	**Lab (self-guided)**	Measure one OpenCode session per turn, write a handoff, resume from it in a fresh session; handout CIDM6333_Lab_E6_ContextBudget.md

The window is an artifact with a budget

On every turn, the model knows what is in the window and nothing else

What the model knows this turn is in the window. Nothing else exists for it. Accidental content still costs attention.

Two rules the spine returns to

Measure every turn: the provider's count against a declared budget. What must survive goes to a file: ledger for decisions, canon for rules.

Tonight's claim, which Sortie 6 will try to disprove

A handoff lets a fresh session resume: its first action is the one the handoff named. Falsified if the fresh session asks a question the handoff answered, an L3 view overrules the ledger, any turn lacks a budget record, or the old transcript was pasted.

The window's contents, and who put them there

Six categories; each one grows for a different reason, and compaction treats each one differently

Category	Who puts it there	Grows with	After /compact in Claude Code (documented)
System prompt, tool definitions	The harness	Tools enabled; MCP servers (names only while deferred)	Reloaded
Instruction files (AGENTS.md, CLAUDE.md, rules)	You	Every line you add; imports load in full	Reloaded from disk
Skill descriptions	You, plugins	Skills installed	Listing not reloaded; invoked skill bodies re-injected, 5,000 tokens each at most
Conversation history	You and the model	Every turn	Replaced by one structured summary
Tool results (reads, test output, pages)	The tools	Every call; one log file can be tens of thousands of tokens	Full outputs gone; up to five recently modified files re-read
Retrieved memory and notes	Your code	What you choose to retrieve	Auto memory reloaded; the rest only if your code reloads it

Source: Claude Code docs, 'Explore the context window' and 'Commands', checked 09/24/2026. Codex and OpenCode publish no per-category list; measure theirs.

The last column matters for Sortie 6: what you said only in conversation becomes a summary.

Why a budget below a million-token window

The window's size is the provider's limit; your budget is a design decision below it

18

models tested (Chroma, Jul 2025); all less reliable as input grew

middle

where long-context accuracy is lowest (Liu et al., TACL 2024)

n^2

token-pair relations: the 'attention budget' (Anthropic, Sep 2025)

84%

fewer tokens with context editing, 100-turn eval (Anthropic, 2025)

Declare a turn budget below the window; past 80%, clear, compact, delegate, or hand off. Big Pickle's window is unpublished: declare and hold a budget.
Anthropic (Sep 2025): 'the set of strategies for curating and maintaining the optimal set of tokens (information) during LLM inference.'

Declare the budget before you fly. Log the number the provider billed, not an estimate.

Instrument the window: one budget record per turn

Read the count from the provider's response; log it to an append-only file; act on it in the loop

In your loop, after every model call

```
u = resp.usage   # the provider's count, not your estimate
total = u.prompt_tokens + u.completion_tokens
rec = {"event": "turn_budget", "turn": t,
       "in": u.prompt_tokens, "out": u.completion_tokens,
       "total": total, "budget": BUDGET,
       "pct": round(100 * total / BUDGET)}
ledger.append(rec)   # one JSON object per line, never edited
if total > 0.8 * BUDGET:
    history = compact(history, keep=KEEP)
```

Field names differ: OpenAI-compatible endpoints (Ollama's /v1, most gateways) return prompt_tokens and completion_tokens; Anthropic's API returns input_tokens, output_tokens, and cache fields counted separately.

What each record must answer

- Which turn and session.
- Tokens in and out, as the provider counted them.
- The declared budget and the percentage used.
- By category where you can; label estimates.
- What the loop did: nothing, clear, compact, delegate, hand off.
- The record is L2; the counter is L3.

If the provider returns no usage, declare a fallback count before Go; discovered in flight, ABORT.

Compaction: four moves, from cheapest to most lossy

Decide what compaction may lose before it runs

1 → 2 → 3 → 4

Clear

Drop old tool results; keep the record of the call. Anthropic: tool result clearing; OpenCode: compaction.prune. Loses raw output only.

Summarize

A model writes a structured summary: objective, decisions, files, errors, pending work. Lossy by design; the summarizer chooses what is lost unless you do.

Keep verbatim

A declared keep list survives untouched: objective, open test point, last few turns, pointers to canon. You write the list.

Re-seed from disk

After the summary, reload L1 files and the ledger tail. Claude Code reloads CLAUDE.md and re-reads up to five recently modified files.

Knobs: Claude Code /compact [focus], /autocompact, PreCompact hook (exit 2 blocks) · Codex /compact, model_auto_compact_token_limit, compact_prompt · OpenCode /compact, compaction.auto / prune / reserved

Progressive disclosure and isolation: load later, or elsewhere

Every technique trades tokens now for a lookup later; the lookup can fail, so log it

Skills: the description now, the body on demand

Name and description sit in context; the body loads on invocation (OpenCode's skill tool; Claude Code). E5 moved procedures into skills; tonight you count the saving.

Deferred tools

Only MCP tool names enter context until a task needs one (Claude Code default). Cursor's A/B test (Jan 2026): 46.9% fewer agent tokens.

Just-in-time retrieval

Keep identifiers, not contents: a path, a query, a ledger event id. Load content at runtime (Anthropic, Sep 2025). A handoff points at evidence.

Subagents: a fresh window

Verbose work stays in the child's window; a summary returns, often 1,000–2,000 tokens (Anthropic). The summary is a claim, not evidence.

Anthropic calls this 'just in time' context. LangChain's four verbs (Jul 2025): write, select, compress, isolate.

Handoffs: writing for a stranger with no transcript

The seven sections from Chapter 15, and the machinery that makes a handoff load before turn one

The seven sections

1 Current state; commit hash.
2 Important context. What a stranger gets wrong.
3 Decisions, with attribution.
4 Next steps; the first is one action.
5 Pending work; blocked, and on what.
6 Verification checklist; commands that prove the state.
7 Evidence; pointers, not pastes.

The machinery around it

Session-start advisement. The harness reads the handoff before turn one (Claude Code: SessionStart hook).
Rulings store. Settled decisions, append-only; never re-asked.
Exit bookmark. The last action and the resume point.
In the wild. Anthropic's harness (Nov 2025): progress file, JSON feature list, git log.

The test: a stranger with no transcript takes the action you named and asks nothing you answered.

Layers of truth: canon, ledger, derived

Declare each layer's files in your README, and the precedence among them

Layer	What it is	In your system (example paths)	Rule
L1 canon	Markdown written by people: PRD, ADRs, AGENTS.md, flight cards, the handoff template	docs/prd.md · docs/adr/ · AGENTS.md · evolution-*/flight-card.md	Changes by commit and review
L2 ledger	Append-only events the system wrote when things happened	var/ledger.jsonl: turn_budget, compaction, handoff_written, session_resumed	Never edited; a correction is a new event
L3 derived	Views computed from L1 and L2: status pages, summaries, frontmatter, markers, on-screen counters	status.md · the compaction summary · a drafted handoff · the TUI's token counter	Regenerable; can never block a gate

Precedence L2 > L1 > L3. 'Frontmatter can lie; the ledger is the truth.' (gzkit state doctrine, docs/governance/state-doctrine.md, ADR-0.0.9)

Where does a compaction summary sit? L3. It may inform the next turn; it may not overrule the ledger.

Common mistakes: three ways a sound-looking build fails

Each one has passed a demo and failed a sortie

“The chat is the memory”

A rule typed in conversation lives in history, which compaction summarizes. Claude Code reloads CLAUDE.md after /compact, not what you typed.

Fix. Put the rule in canon (L1) or the ledger (L2).

App. C #5 correction fails

“The counter is the window”

File size, or a session total, reported as context. OpenCode's session totals add every turn's cache reads (issue #30649).

Fix. Log per call from the provider's usage fields.

App. C #4 skipped cheap verification

“The handoff is a diary”

‘Made good progress on the loop; continue tomorrow.’ No next action, no names, no evidence. The fresh session guesses.

Fix. Seven sections; the first next step is a command.

Human-side: summary laundering

Each fix moves a fact out of the window and into a file the next session reads on purpose.

Break · 0:55–1:00

Five minutes

If your coding session died now, what in its window could you not rebuild from files?

Teardown: four context surfaces side by side

Where the instructions come from, how you see the window, when it compacts, how it keeps things out

Surface	Claude Code	Codex CLI	Cursor	OpenCode (yours)
Instruction file	CLAUDE.md hierarchy; reads AGENTS.md itself since v2.1.277 when no CLAUDE.md	AGENTS.md root → cwd, concatenated; stops at 32 KiB (project_doc_max_bytes)	.cursor/rules (.mdc: always, intelligent, globs, manual); nested AGENTS.md	AGENTS.md walking up; CLAUDE.md as fallback; first match wins
See the window	/context: grid by category, with warnings	/status: tokens used and context remaining		TUI counters; opencode export JSON; opencode stats
Compaction	Auto at the auto-compact window (/autocompact); /compact [focus]; PreCompact hook	Auto at model_auto_compact_token_limit; /compact; compact_prompt override	Summarizes when full; history kept as a file the agent can search	compaction.auto when full; reserved buffer; prune old tool output; /compact
Keep it out	Skills on demand; MCP deferred; subagents in fresh windows	tool_output_token_limit (sample: 12,000 per output)	Tool output, terminals, MCP tools as files	Skill tool; explore and general subagents in child sessions

Sources checked 09/24/2026: code.claude.com docs (memory, commands, model-config, hooks); developers.openai.com/codex (slash commands, config sample, AGENTS.md guide); cursor.com docs and blog (Jan 6, 2026); opencode.ai docs (config, TUI, CLI, agents, skills).

Teardown: Claude Code, read for mechanism

The documented list of what survives /compact is the design, stated

Mechanism: what comes back after /compact

- System prompt, CLAUDE.md, auto memory, MCP tool names: reloaded.
- Up to five recent files: re-read.
- Invoked skills: re-injected, 5,000 tokens each.
- Skill listing: not reloaded.
- Conversation: one structured summary; tool outputs and reasoning gone.
- Auto-compaction: 1M-window Sonnet 5 compacts at about 967K by default.

Choice · cost · failure mode

Design choice. Files reload; conversation is summarized. Write what matters down.
Cost. Detail is gone; /compact is itself a large request.
Failure mode #5: a chat-only correction survives as paraphrase.
Control points. /compact Focus on …; 'Compact instructions' in CLAUDE.md; PreCompact hook; SessionStart matcher compact.

The harness already implements rule two: files reload, chat does not. Your loop should too.

Teardown: Codex, read for mechanism

A byte cap on instructions and a compaction you cannot read

Mechanism

AGENTS.md from the Git root down to the working directory, concatenated; closer files override. Reading stops at 32 KiB by default.

/compact summarizes; auto-compaction at model_auto_compact_token_limit. On the Responses API, compaction returns an encrypted item passed back as is.

Choice and cost

Choice. A hard byte cap on instructions, and server-side compaction that keeps reasoning state the client never sees.
Cost. Files past 32 KiB are not read; the most specific guidance goes first. The compaction item is 'opaque and not intended to be human-interpretable'.

Failure mode

Human-side: mediated observation. The Chase cannot read the summary; a pass rests on what the model says it remembers.

Design Analysis question. If your record of what was kept is unreadable, where does evidence of intent live? Your own ledger.

Sample config: # model_auto_compact_token_limit = 64000 · # tool_output_token_limit = 12000 · project_doc_max_bytes default 32768

Teardown: Cursor, read for mechanism

Dynamic context discovery (Jan 6, 2026): the file system is the interface to context

Mechanism

Long tool outputs, terminals, and MCP tool descriptions become files; names first. During summarization the agent searches the chat history as a file.

Rules

Project rules in .cursor/rules as .mdc: Always Apply, Apply Intelligently, Apply to Specific Files (globs), Apply Manually. 'Keep rules under 500 lines.' Nested AGENTS.md applies.

Choice and cost

Choice. Nothing is dropped; it moves to disk for the agent to grep. 46.9% fewer tokens in MCP-tool runs.
Cost. Every recovery is a tool call the model must think to make.

Failure mode

'Apply Intelligently' loads a rule only when the model judges it relevant. An every-turn rule placed there is wishful (E5 Probe 2). App. C #4: the cheap lookup skipped.

Teardown: OpenCode, the harness you fly

What it lets you configure, what it lets you measure, and the question it leaves open

Configure (opencode.json, v1.18.x)

```
{
  "compaction": {
    "auto": true,
    "prune": false,
    "reserved": 10000
  }
}
```

- auto: compact when context is full (default true).
- prune: remove old tool outputs (default false).
- reserved: token buffer so compaction itself does not overflow.
- /compact (alias /summarize) by hand; a hidden compaction agent writes the summary.
- v2 (beta) renames these keys; check your version's docs.

Measure, and the open question

Measure. No /context grid: TUI counters, opencode stats, opencode export <session-id> (token counts per assistant message).
Isolate. Explore and general subagents run in child sessions.
Open question. Auto-compaction fires at model limit minus output reserve. Big Pickle's window is unpublished: what limit does OpenCode use?

A free model with an unpublished window: the only budget you can trust is the one you declared and logged.

Teardown: Anthropic's harness for long-running agents

Young (Anthropic Engineering, Nov 26, 2025): the handoff built into the harness

> *“Compaction isn’t sufficient.” Every session starts as a stranger.*

Mechanism. An initializer writes init.sh, claude-progress.txt, and a 200+ feature JSON list. Each session reads them, works one feature, commits.
Choice. JSON over Markdown: overwritten less. Cost: one feature per session.
Failure mode. App. C #9, metagaming: flip a status, never rewrite the list.

Live Design Analysis: Claude Code's context system

The nine components on the board; most of the words go to memory and context, governance, and coordination

Component	Answer (we fill the gaps together)
Foundation	Claude models with 200K to 1M windows; auto-compact window tuned per model, adjustable 100K–1M
Perception	User turns, file reads, tool results, hook additionalContext, SessionStart output
Planning & reasoning	Extended thinking; plan mode before edits
Tools & orchestration	Built-in tools; MCP schemas deferred behind tool search; skills listed, bodies on demand
Memory & context	CLAUDE.md hierarchy; auto memory (first 200 lines or 25KB); /context; files reload, chat does not
Coordination	Subagents in fresh windows; a summary returns; nesting up to three layers by default
Evaluation & feedback	/context warnings; /usage prompt-cache line counts misses and expected rebuilds after compaction
Governance & human interface	Permissions; hooks: PreCompact can block (exit 2), SessionStart can re-inject after compaction
Runtime & operations	Terminal, IDE, cloud; prompt cache 5-minute or 1-hour lifetime; a compaction rebuilds the cache

Judgment loop: Decision (files reload, chat summarized) · Characteristics privileged (a small, fresh window) · Cost (detail; an expensive compaction call) · Failure mode (#5) · Preservation (the reload list; the hooks).

Reference design: the state doctrine and the handoff

gzkit's three layers, and the handoff and rulings machinery built on them

state-doctrine.md (ADR-0.0.9)

- L1 canon: Markdown (ADRs, rules, AGENTS.md).
- L2 ledger: append-only events, the runtime truth.
- L3 derived: status, markers, frontmatter.
- Precedence L2 > L1 > L3; L3 blocks no gate.
- 'Frontmatter can lie; the ledger is the truth.'
- Which of your files is L3, treated as truth?

Handoffs, rulings, orientation

- gz handoff create / resume: the seven-section handoff.
- .gzkit/handoffs/rulings.jsonl: settled rulings, append-only.
- scripts/session_orientation.py: what a session reads before turn one.
- By 2026-06-20, 116 handoffs; the 'utter shambles' audit counted them.

The ledger outranks the prose that describes it, and both outrank anything computed from them.

Reference design: the stale clone, 2026-04-26

The session's picture of the repository was a derived view, and nobody checked its freshness

10

commits behind: the clone a session oriented itself on

~215

lines discarded by the hard reset that reconciled it

1

check added at SessionStart: ahead/behind, before the first turn

What happened. A clone ten commits behind; a hard reset discarded about 215 lines.
What resulted. A SessionStart ahead/behind check before any work; orientation states its freshness.
For your system. The handoff names a commit; resume checks the tree is there first.

Source: gzkit incident casebook (Appendix G).

Reference design: three more episodes about context

Each one turned a context failure into a rule

2026-08-23 · settled rulings re-asked

An agent re-elicited three settled rulings by multiple choice. Rule: 'A settled question is a drift vector.' Settled answers load at session start from the rulings store.

Your handoff's Decisions section

2026-05-23 · four questions in one prompt

Four questions bundled into one prompt. Result: operator economy of effort, one question at a time, only questions the record cannot answer.

Your fresh session's first turn

AGENTS.md as a derived view

AGENTS.md rendered from the corpus (ADR-0.0.37), 'lowest amount of faith in AGENTS.md'; map, not encyclopedia (ADR-0.0.54) after the 258K exhaustion.

Every one of the three is the same move: a fact the session kept asking for, written down where the next session reads it first.

Evaluation lens: a strong Sortie 6

The Chase reads the budget log, the ledger diff, the handoff, and the fresh session's first two turns

Weak

- Budget reported as file size or a session total
- Compaction at harness default; no event, no keep list
- Handoff is a paragraph: no next action, no names
- Resume 'tested' by pasting the old transcript
- README says 'the ledger is the truth' with no paths

Sound

- One budget record per turn, from provider usage fields
- Compaction with a keep list; tokens before and after logged
- Seven-section handoff; first next step is one action
- Fresh session's first action matches; nothing re-asked
- L1/L2/L3 with paths; a planted stale L3 loses to the ledger

Exceptional

- Budget by category; one category cut, before and after
- Compaction summary checked against the ledger; dropped decision pinned
- Handoff generated from the ledger (L3 of L2); fidelity checked
- A settled ruling stays settled; not re-asked
- Same handoff resumed on a second model family; difference explained

Flight Card 6 brief: Sortie 6, the handoff

Expansion · depends on Sortie 5 PASS · Go is asynchronous: the Chase signs in the repo before you fly

```
ENVELOPE            expansion
DESIGN CLAIM        A handoff written at session end lets a fresh session resume with no loss of intent:
                    the fresh session's first action is the one the handoff named.
SYSTEMS UNDER TEST  budget meter · compactor + keep list · handoff writer · resume loader · runtime ledger
                    · layers-of-truth declaration (README)
SUBSTRATE           repo + commit · model/route + declared turn budget · harness + version
DEPENDS ON          Sortie 5 PASS (server hardened; AGENTS.md under budget and measured)

TEST POINTS
  1  session A runs a bounded task, >= 6 turns   → one turn_budget record per call {turn, in, out, total, budget}
  2  context crosses the compaction threshold     → compaction event {before, after, kept[]}; objective and L1
                                                     pointers present after; task continues
  3  session A ends                               → handoff.md (7 sections) + handoff_written event with hash
  4  fresh session B loads handoff + L1 + ledger  → first action == handoff NEXT STEP 1; zero questions the
     (no transcript)                                 handoff answered; session_resumed event
  5  planted stale L3 view contradicts ledger     → B follows the ledger; conflict logged; L3 blocks nothing

INSTRUMENTATION     budget log (every call) · ledger diff (compaction, handoff_written, session_resumed)
                    · handoff.md + hash · B's first two turns captured verbatim

PRE-REGISTERED FALSIFIERS (any ONE ⇒ FAIL)
  - B asks a question the handoff answered, or takes a first action the handoff did not name
  - an L3 view contradicts the ledger and wins
  - any call with no budget record, or a compaction with no event
  - passing by pasting A's transcript into B, or by editing the handoff after B starts

ABORT (halt-not-fail)  provider returns no usage fields and no fallback count declared; model unavailable
CHASE               reads budget log, ledger diff, handoff.md, B's first two turns; never the author's summary
GO/NO-GO            Chase signs GO in the repo by the course-calendar date; no sortie before GO

SPRINT              Sortie 6 log · budget + compaction + handoff + ledger · layers in README · DA · AI_LOG
```

Lab tonight, self-guided: measure a session, resume it

Handout: CIDM6333_Lab_E6_ContextBudget.md · OpenCode + Big Pickle or a Go model · no new installs

Thirty minutes, five steps

0–3 Set up evolution-06/lab/; declare a turn budget.
3–10 Session A: read-only, five or six turns; token claim.
10–17 opencode export <id>; budget.py; per-turn table.
17–25 Write handoff.md by hand, seven sections.
25–29 Session B (/new) reads only the handoff; match?
29–30 Commit; ledger event.

What you commit

- evolution-06/lab/budget.md: the per-turn table, declared budget, turns over budget.
- evolution-06/lab/handoff.md.
- evolution-06/lab/resume-check.md: B's first action and questions, verbatim, and your verdict.
- One line: the model's claim against the measured number.
- Do not commit the raw export; it contains file contents.

Tonight's session is OpenCode's window, not your system's; the Sprint moves the same meter into your own loop.

Homework: the Sprint

Brief: CIDM6333_HW_E6_ContextEngineering.md · Go and Chase are asynchronous and in the repo

Hand in

- Sortie 6 flight log
- Budget meter, compactor and keep list, handoff writer, resume loader, ledger; tests
- README: layers of truth, with paths and precedence
- Design Analysis: Claude Code, Codex, Cursor, or OpenCode context system
- AI_LOG entry

Go/No-Go, before you fly

Commit evolution-06/flight-card.md by the date in the course calendar. Your Chase signs GO or NO-GO with one reason. No signature, no sortie. A NO-GO names the field to fix.

Chase review, after you fly

The Chase reads the budget log, the ledger diff, handoff.md, and B's first two turns; nothing you wrote about them. One-line verdict, PASS or FAIL, with the falsifier quoted.

FAIL is the claim disproved. ABORT is the flight not flown for an unrelated reason. Log both honestly.

Intent that lives only in the window dies with the window.

Measure every turn, decide what compaction may lose before it runs, and hand off to a stranger. Lab: measure one session and resume it from your own handoff.